



Introduction to Java

and

OOPs Concepts

LEARN | PRACTICE | BUILD | GROW

Write
Compile
Run
Repeat...



What is Java?

- Java is a high-level, object-oriented, platform-independent programming language.
- Developed by Sun Microsystems (now owned by Oracle).
- Well-suited for building web, desktop, mobile and enterprise applications.
- Known for its simplicity, security, robustness and portability.



Key Features of Java



Simple



Secure



Robust



Platform Independent



Object-Oriented



High Performance



Multithreaded



Automatic Memory Management



Rich Standard Library

OOPs Concepts in Java

Object-Oriented Programming (OOP) is a programming paradigm that uses "objects" to design applications. It helps in writing modular, reusable and maintainable code.

1 Encapsulation



Wrapping data (variables) and methods into a single unit (class) and restricting direct access to the data.

Helps in data hiding and security.

2 Abstraction



Hiding the implementation details and showing only essential features to the user.

Helps in reducing complexity.

3 Inheritance



A mechanism where one class (child) acquires the properties and behavior of another class (parent).

Promotes code reusability.

4 Polymorphism



The ability of an object to take many forms. Achieved through method overloading and method overriding.

Provides flexibility.



Why Learn Java?

- Widely used in industry
- Strong community support
- Used in web, mobile, cloud and enterprise solutions
- Great for learning OOPs and software development fundamentals
- High demand in the job market



Build a Better Tomorrow with Java

“

Simple Language.
Powerful Possibilities.

”



Java Introduction

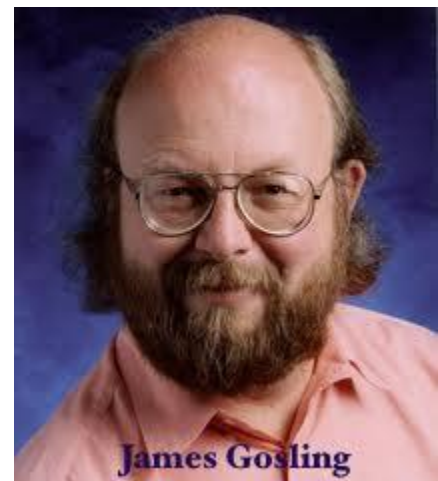
Java is a **programming language** and a **platform**. Java is a high level, robust, object-oriented and secure programming language.

Java was developed by **Sun Microsystems** (which is now the subsidiary of Oracle) in the year 1995. **James Gosling** is known as the father of Java. Before Java, its name was **Oak**. Since Oak was already a registered company, so James Gosling and his team changed the **Oak** name to **Java**.

Platform: Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

History of Java

The history of Java is very interesting. Java was originally designed for interactive television, but it was too advanced technology for the digital cable television industry at the time. The history of Java starts with the **Green Team**. Java team members (also known as Green Team), initiated this project to develop a language for digital devices such as set-top boxes, televisions, etc. However, it was suited for internet programming. Later, Java technology was incorporated by **Netscape**.



The principles for creating Java programming were "Simple, Robust, Portable, Platform-independent, Secured, High Performance, Multithreaded, Architecture Neutral, Object-Oriented, Interpreted, and Dynamic".

Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc. There are given significant points that describe the history of Java.

1. **James Gosling**, **Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.
2. Initially designed for small, **embedded systems** in electronic appliances like set-top boxes.
3. Firstly, it was called "**Greentalk**" by James Gosling, and the file extension was dot .gt.
4. After that, it was called **Oak** and was developed as a part of the Green project.

Java First Program

```
class Simple
{
    public static void main(String args[])
    {
        System.out.println("Hello Java");
    }
}
```

NOTE:

- ✓ **class** – class is a keyword used to create class in Java.
- ✓ **Simple** – It a class name
- ✓ **public static void main(String args[])** – It is main method.
- ✓ **System.out.println("Hello Java")** – This function is used to display user defined message on the screen.

To Write Program

You can use any text editor to write java code such as **Notepad, VS Code, IntelliJ and Eclipse.**

Save Java File

Save the java file with extension **.java**. The location of java file should be bin folder of installed java compiler location.

You can also save java file to your own location such as your own folder such as **C: D: or E:** drive.

NOTE: - If you save your java file outside the bin folder of java compiler location such as your own folder in any drive you have follow the given path:

path=%path%;C:\Java\Jdk1.7.0\bin

Syntax – classname.java

Ex – First.java (Where **First** is the class name)

To Compile Java Program

The name of the compiler of java is **javac**.

Syntax – **javac** classname.java

Ex – **javac** First.java

To Run Java Program

We know java is an interpreter based language, the name of the java interpreter is **java**.

Syntax – java classname

Ex – java First.java

Features of Java

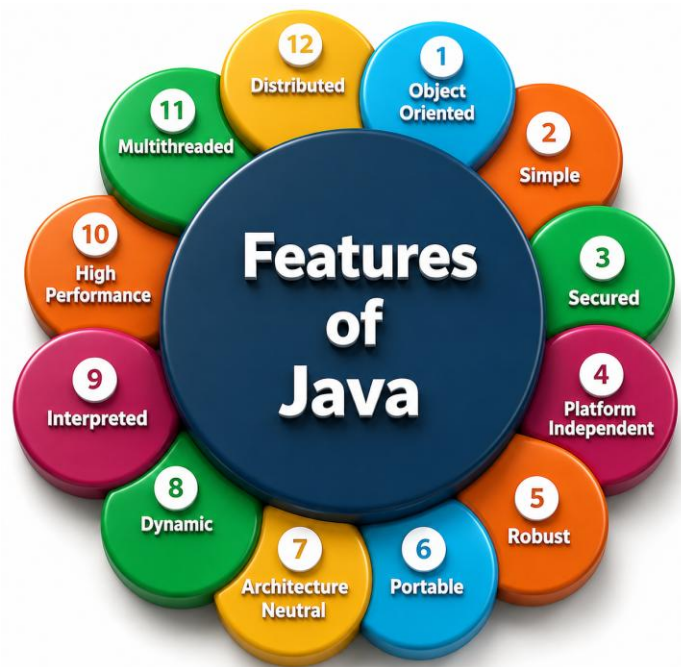
The primary objective of creating the **Java** programming language was to make it portable, simple, and secure. In addition to this, several excellent features contribute significantly to the popularity of this language. These features are often referred to as **Java buzzwords**.

Below is a list of the most important features of the Java language.

Simple

Java is easy to learn, with a syntax that is simple, clean, and easy to understand. According to Sun Microsystems, Java is a simple programming language because:

- ✓ Java syntax is based on C++, making it easier for programmers to learn after C++.
- ✓ Java has eliminated many complex and seldom-used features, such as explicit pointers and operator overloading.



- ✓ There is no need to remove unreferenced objects because Java has Automatic Garbage Collection.

Object-Oriented

Java is an object-oriented programming language where everything is an object. Object-oriented programming means that we organize our software as a combination of various types of objects that incorporate both data and behaviour.

Basic Concepts of OOPs are –

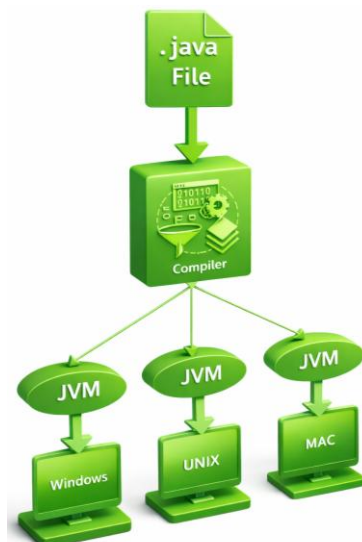
Class, Object, Inheritance, Polymorphism, Inheritance, Abstraction



Portable

Java's portability allows programs to run on various platforms without modification. This is achievable because Java code is compiled into bytecode, which is independent of the underlying hardware and operating system. The JVM on each system interprets this bytecode, ensuring platform compatibility.

Java is portable because its programs are compiled into bytecode, which is independent of hardware and operating systems. The JVM on each platform interprets this bytecode, allowing the same program to run across different environments without modification.

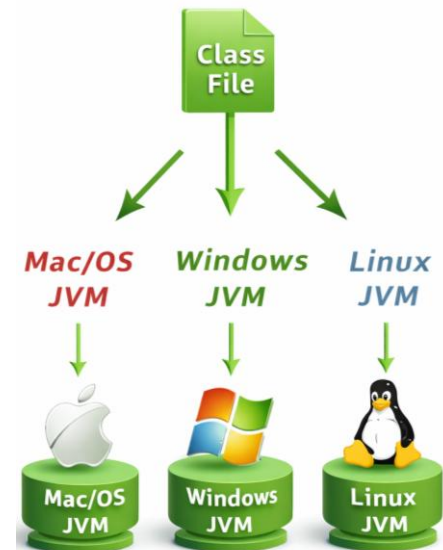


Platform Independent

There are two types of platforms: software-based and hardware-based. Java offers a software-based platform.

The Java platform differs from most other platforms in that it is a software-based platform running on top of hardware-based platforms. It consists of two components:

- ✓ Runtime Environment
- ✓ API (Application Programming Interface)

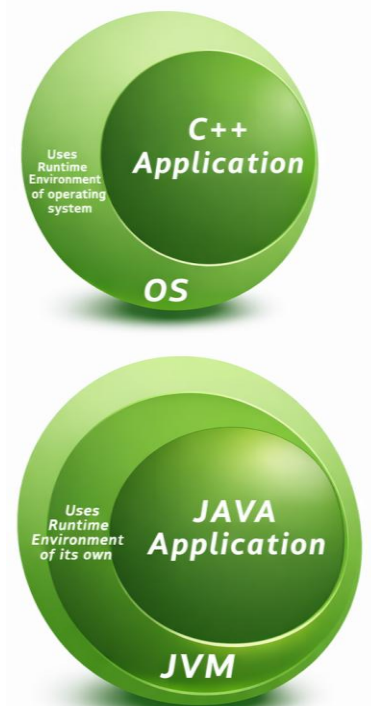


Secured

Java is renowned for its security. With Java, we can create virus-free systems. Java is secure because:

- ✓ No explicit pointer
- ✓ Java Programs run inside a virtual machine sandbox

Java is considered secure because it avoids explicit pointers, reducing risks of memory access errors, and runs programs inside a virtual machine sandbox. This controlled environment ensures that malicious code cannot directly harm the underlying system, making Java applications safer and more reliable.



Robust

The English meaning of **Robust** is strong. Java is robust for the following reasons:

- ✓ It employs strong memory management.
- ✓ There is a deficiency of pointers that prevent security issues.
- ✓ Java offers automatic garbage collection, which operates on the Java Virtual Machine to dispose of objects that are no longer being utilized by a Java application.
- ✓ Java includes exception handling and type checking mechanisms. All these factors contribute to making Java robust.

Architecture-Neutral

Java is architecture-neutral because it lacks implementation-dependent features; for instance, the size of primitive types is fixed.

In C programming, the int data type occupies 2 bytes of memory in a 32-bit architecture and 4 bytes in a 64-bit architecture. However, it occupies 4 bytes of memory in both 32-bit and 64-bit architectures in Java.

Interpreted

Java's interpreted feature means that Java code is not directly converted into machine code by a compiler. Instead, first, it is compiled into bytecode, then executed by the JVM through an interpreter. It allows Java to be platform-independent, meaning the same bytecode can run on any system with a JVM.

The below figure shows the sequential execution of java program.



Sequence of Java Program Execution

1. Java Source Code (.java)

- ✓ You write your program in a .java file using Java syntax.
- ✓ Example: HelloWorld.java.

2. Compilation with javac

- ✓ The Java compiler (javac) translates the source code into bytecode.
- ✓ Output: .class file.

3. Class Loader

- ✓ The JVM's Class Loader loads the .class file into memory.
- ✓ It organizes classes and ensures they are available for execution.

4. Bytecode Verification & Transformation

- ✓ The loaded class is checked for security and correctness.
- ✓ It may be transformed or optimized before execution.

5. Interpreter / JIT Compiler

- ✓ The JVM interprets the bytecode line by line.
- ✓ Alternatively, the Just-In-Time (JIT) compiler converts bytecode into native machine code for faster execution.

6. Executing Code

- ✓ The program logic runs as intended (loops, conditions, method calls).
- ✓ This is where your application actually performs tasks.

7. Machine Code

- ✓ Finally, the JVM translates instructions into binary machine code (0s and 1s).
- ✓ The CPU executes these instructions directly.

High-Performance

Java is faster than traditional interpreted programming languages because Java bytecode is "close" to native code. However, it is still slightly slower than compiled languages, such as C++. As an interpreted language, Java is slower than compiled languages like C and C++.

Multithreaded

A thread is like a separate program that executes concurrently. We can write Java programs to handle many tasks simultaneously by defining multiple threads. The main advantage of multi-threading is that it does not occupy memory for each thread but rather shares a common memory area. Threads are crucial for multimedia, web applications, and more.

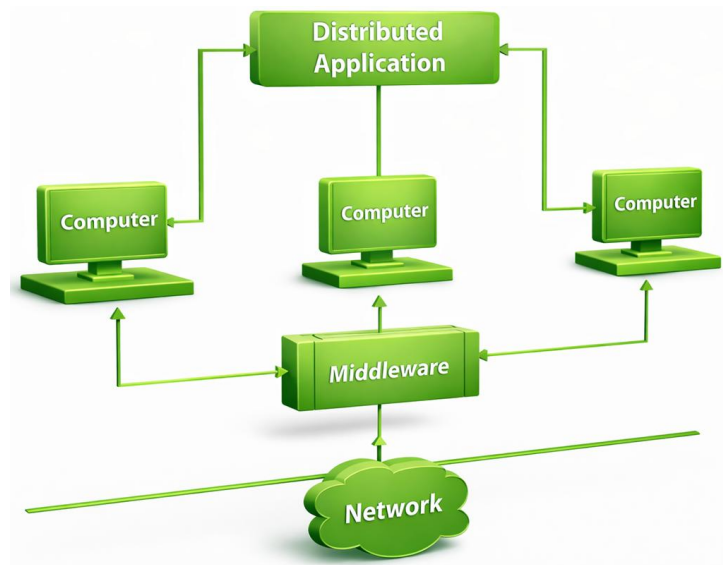
Dynamic

Java is a dynamic language that supports the on-demand loading of classes. It also allows the use of functions from its native languages, namely C and C++.

Java supports dynamic compilation and automatic memory management, including garbage collection.

Distributed

Java is designed for distribution, allowing users to develop distributed applications effectively. **RMI** and **EJB** facilitate the creation of these distributed applications. This feature enables access to files by invoking methods from any machine on the Internet.



Important Points to Remember

The following points capture the essence of the standout features and advantages of Java:

- ✓ Java is platform agnostic; this implies that programs authored on any system can be executed on any other system that has a JVM.
- ✓ The object-oriented paradigm is fully supported IT Java, including inheritance, encapsulation, abstraction, and polymorphism.
- ✓ Memory can be managed automatically through the efficient process of Automatic Garbage Collection.
- ✓ No explicit pointers mean added security that simplifies processes.
- ✓ Portability and architectural neutrality are supported, whereby Java applications can be compiled once and run in multiple locations due to Java bytecode.
- ✓ Exception handling makes sure programs can anticipate problems and resolve them without terminating the application while running.
- ✓ Java supports multithreading, enabling the execution of multiple threads simultaneously to build efficient and responsive applications.

Now, Definition of Java can be:

Java is simple, object oriented, portable, platform independent, secure, robust, architecture – neutral, interpreted, high performance, multithreaded, distributed and dynamic language.



Introduction

Object-Oriented Programming (OOPs) is a fundamental programming paradigm in Java that organizes software design around data, or objects, rather than functions and logic.

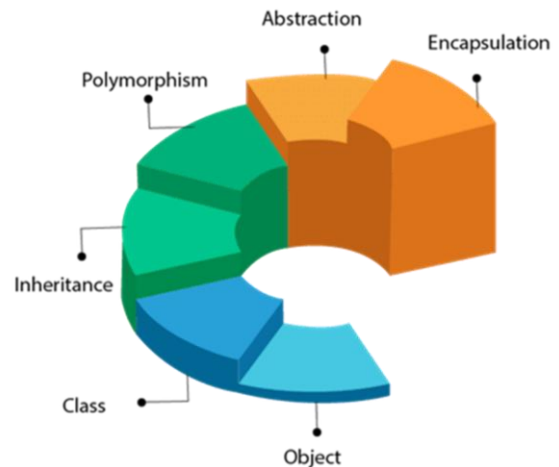
Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects that contain data (fields) and behavior (methods). It focuses on designing software that closely represents real-world entities. It is used to:

- ✓ Improves code reusability
- ✓ Enhances maintainability and scalability
- ✓ Makes programs easier to understand and manage

- ✓ Closely models real-world entities

OOPS Characteristics

- ✓ Class
- ✓ Object
- ✓ Encapsulation
- ✓ Inheritance
- ✓ Polymorphism
- ✓ Abstraction



Class

A Class in Java is a blueprint or template used to create objects. It defines the properties (data) and behaviors (methods) that objects of that class will have.

- ✓ A class is a user-defined data type.
- ✓ It contains variables (fields) and methods.

Multiple objects can be created from a single class

Analogy: A house blueprint drawn on paper. You cannot live in it, but it shows how to build the house.

Example:

- ✓ **Class** – Blueprint of a car
- ✓ **Fields** – Car's specifications (color, speed)
- ✓ **Method** – Car's behavior (drive)
- ✓ **Object** – Actual manufactured car

```
// This is the blueprint
class Car
{
    String color;
    int speed;
    void drive()
    {
        System.out.println("The car is moving!");
    }
}
```

Example: A Car represents a class (blueprint), while BMW, Mercedes, and Audi represent objects (instances) created from that class.



Object

Any entity that has state and behaviour is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An Object can be defined as an instance of a class. It contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.

An Object in Java is an instance of a class that represents a real-world entity. It is used to access the variables and methods defined inside a class.

- ✓ **State:** Represents the current data or attributes of an object.
- ✓ **Behavior:** Represents the actions that an object can perform.
- ✓ **Identity:** Every object has a unique identity in memory that distinguishes it from other objects, even if they contain the same data.

Analogy: The actual physical car built from the blueprint. It has real body and a real color.

```
public class Main
{
    public static void main(String[] args)
    {
        // Creating an object named 'myCar' from the Car class
        Car myCar = new Car();
        myCar.color = "Red"; // Giving it data
        myCar.drive();      // Making it do an action
    }
}
```

Encapsulation

Encapsulation is the process of wrapping data and methods into a single unit, usually a class, and restricting direct access to the data. It acts as a protective shield that prevents data from being accessed directly from outside the class.

- ✓ Data members are hidden using the private access modifier.
- ✓ Access to data is provided through public getter and setter methods.
- ✓ It improves data security, maintainability, and controlled access.

Analogy: A medical capsule. The medicine is safely hidden inside the plastic shell so you do not spill it.

```
class BankAccount
{
    // 'private' hides the money variable from outside classes
    private double balance = 1000;
    // You must use this safe method to check the balance
    public double getBalance()
    {
        return balance;
    }
}
```

Complete Example to Demonstrate Encapsulation

```
class Car
{
    // Private data members (hidden) Encapsulation
    private String color;
    private int speed;

    // Public getters and setters (controlled access)
    public String getColor()
    {
        return color;
    }
    public void setColor(String color)
    {
        this.color = color;
    }

    public int getSpeed()
```

```

    {
        return speed;
    }

    public void setSpeed(int speed)
    {
        if(speed > 0)
        {
            this.speed = speed;
        }
        else
        {
            System.out.println("Speed must be positive!");
        }
    }

    public void drive()
    {
        System.out.println("The " + color + " car is driving at " +
                           speed + " km/h!");
    }
}

public class EncapsulationDemo
{
    public static void main(String[] args)
    {
        // Object demonstrates encapsulation
        Car myCar = new Car();

        // Accessing data only through methods
    }
}

```

```
myCar.setColor("Blue");  
myCar.setSpeed(100);  
  
System.out.println("Car Color: " + myCar.getColor());  
System.out.println("Car Speed: " + myCar.getSpeed());  
  
myCar.drive();  
}  
}
```

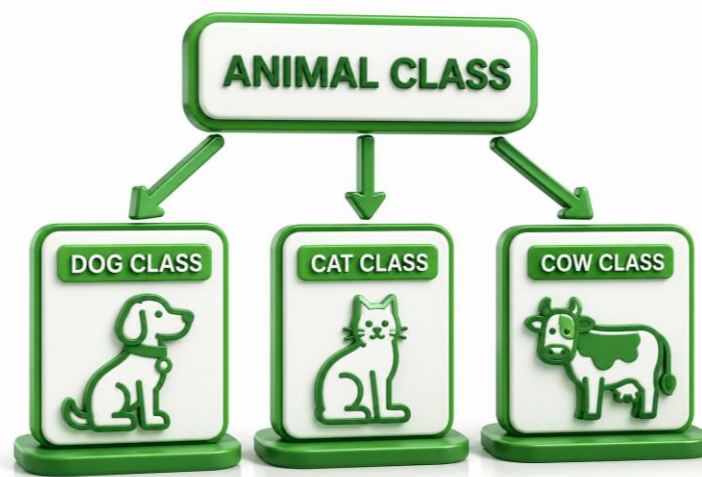
Inheritance

Inheritance is a core OOP concept in Java that allows one class to acquire the fields and methods of another class using the extends keyword. It represents an "is-a" relationship between classes.

- ✓ The class being inherited is called the superclass, and the inheriting class is the subclass.
- ✓ A subclass can use existing features of the superclass and also add its own.

Inheritance promotes code reusability and reduces redundancy.

Analogy: A child inheriting their parents' eye color or last name.



Example:

```

class Animal // Parent Class
{
    void eat()
    {
        System.out.println("This animal eats food.");
    }
}
// Child Class inherits from Animal using 'extends'
class Dog extends Animal
{
    void bark()
    {
        System.out.println("The dog says Woof!");
    }
}
// Note: A Dog object can now use both eat() and bark()!

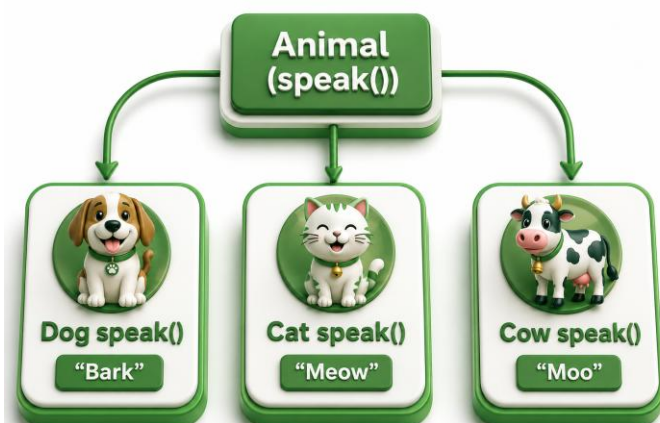
```

Polymorphism

Polymorphism means “many forms”, where a single entity can behave differently in different situations. In Java, it allows the same method or object to show different behavior based on context.

- ✓ Same method, different behavior depending on the object
- ✓ Achieved through method overloading and method overriding

Example: Different animals represent polymorphism, where the same method `Speak()` produces different outputs like Bark, Meow, and Moo depending on the object.



Types of Polymorphism

1. Compile-time Polymorphism (Static Polymorphism)

- ✓ Achieved through method overloading (same method name, different parameter lists).
- ✓ The compiler decides which method to call at compile time.

Example:

```
class MathUtils
{
    // Overloaded methods
    int add (int a, int b)
    {
        return a + b;
    }

    double add (double a, double b)
    {
        return a + b;
    }

    public static void main(String[] args)
    {
        MathUtils obj = new MathUtils();
        System.out.println(obj.add(5, 10));           // calls int version
        System.out.println(obj.add(5.5, 10.5));       // double version
    }
}
```

2. Runtime Polymorphism (Dynamic Polymorphism)

- ✓ Achieved through method overriding (subclass provides its own implementation of a method defined in the parent class).
- ✓ The JVM decides which method to call at runtime based on the object type.

Example:

```
class Car
{
    void drive()
    {
        System.out.println("Car is driving");
    }
}

class SportsCar extends Car
{
    @Override
    void drive()
    {
        System.out.println("SportsCar is driving fast!");
    }
    public static void main(String[] args)
    {
        Car myCar = new SportsCar(); // reference of parent, object
                                     // of child
        myCar.drive(); // calls overridden method at runtime
    }
}
```

Abstraction

Abstraction in Java is the process of hiding implementation details and showing only the essential features of an object. It helps users focus on what an object does rather than how it does it.

- ✓ Hides complexity: Internal implementation details are hidden from the user.

- ✓ Improves maintainability: Changes in implementation do not affect the user code.
- ✓ Enhances flexibility: Supports loose coupling through abstract classes and interfaces.

Example:

// Abstract class

abstract class Vehicle

{

 // Abstract method (no body)

 abstract void start();

 // Concrete method (has body)

 void stop() {

 System.out.println("Vehicle stopped.");

 }

}

// Concrete subclass

class Car extends Vehicle

{

 @Override

 void start()

 {

 System.out.println("Car starts with a key ignition.");

 }

}

// Another subclass

class Bike extends Vehicle

{

```
@Override
void start()
{
    System.out.println("Bike starts with a self-start button.");
}

// Demonstration
public class Abstraction
{
    public static void main(String[] args)
    {
        Vehicle myCar = new Car();    // abstraction in action
        myCar.start();
        myCar.stop();

        Vehicle myBike = new Bike();
        myBike.start();
        myBike.stop();
    }
}
```

All Feature of OOPs in a Single Program

```
// 1. Abstraction: Abstract class defines template
abstract class Vehicle
{
    abstract void start(); // abstract method (must be implemented)
    void stop()           // concrete method
    {
        System.out.println("Vehicle stopped.");
    }
}
```

```
    }  
}  
  
// 2. Inheritance: Car inherits Vehicle  
class Car extends Vehicle  
{  
    private String model; // Encapsulation: private field  
  
    public Car(String model) // Constructor  
    {  
        this.model = model;  
    }  
  
    public String getModel() // Getter & Setter (Encapsulation)  
    {  
        return model;  
    }  
    public void setModel(String model)  
    {  
        this.model = model;  
    }  
  
    // Implement abstract method  
    @Override  
    void start()  
    {  
        System.out.println(model + " starts with key ignition.");  
    }  
}  
  
class Bike extends Vehicle  
{
```

```

@Override
void start()
{
    System.out.println("Bike starts with self-start button.");
}
}

// 3. Polymorphism: Same method behaves differently
public class OOPsExample
{
    public static void main(String[] args)
    {
        // Using abstraction + inheritance
        Vehicle myCar = new Car("Honda City");
        Vehicle myBike = new Bike();

        // Polymorphism: start() behaves differently
        myCar.start();
        myBike.start();

        // Encapsulation: access private field via getter/setter
        Car carObj = (Car) myCar;
        System.out.println("Car model: " + carObj.getModel());
        carObj.setModel("Hyundai Verna");
        System.out.println("Updated Car model: " + carObj.getModel());

        // Common method from abstract class
        myCar.stop();
        myBike.stop();
    }
}

```

OUTPUT

```

Honda City starts with key ignition.
Bike starts with self-start button.
Car model: Honda City
Updated Car model: Hyundai Verna
Vehicle stopped.
Vehicle stopped.

```

Menu Based CLI Program for Calculation

```
import java.util.Scanner;

public class MenuCalculator
{
    private Scanner sc = new Scanner(System.in);
    private int num1, num2;

    // Method to take input
    public void entry()
    {
        System.out.print("Enter first number : ");
        num1 = sc.nextInt();
        System.out.print("Enter second number: ");
        num2 = sc.nextInt();
    }

    // Addition
    public void add()
    {
        entry();
        System.out.println("Sum = " + (num1 + num2));
    }

    // Subtraction
    public void subtract()
    {
        entry();
        System.out.println("Difference = " + (num1 - num2));
    }
}
```

```
// Multiplication
public void multiply()
{
    entry();
    System.out.println("Product = " + (num1 * num2));
}

// Main Menu Loop
public void showMenu()
{
    int choice;
    do {
        System.out.println("-----MAIN MENU-----");
        System.out.println("1. Addition\t\t2. Subtraction");
        System.out.println("3. Multiplication\t\t0. EXIT");
        System.out.println("-----");
        System.out.print("Enter Your Choice: ");
        choice = sc.nextInt();

        switch (choice)
        {
            case 1:
                add();
                break;
            case 2:
                subtract();
                break;
            case 3:
                multiply();
                break;
            case 0:

```

```
        System.out.println("Exiting Calculator...  
                                Goodbye!");  
  
        break;  
    default:  
        System.out.println("Invalid choice! Please try  
                                again.");  
    }  
    System.out.println(); // blank line for readability  
} while (choice != 0);  
}  
  
// Main Method  
public static void main(String[] args)  
{  
    MenuCalculator calc = new MenuCalculator();  
    calc.showMenu();  
}  
}
```

OUTPUT

```
-----MAIN MENU-----  
  
1. Addition                2. Subtraction  
3. Multiplication          0. EXIT  
  
-----  
  
Enter Your Choice:
```

Conclusion

By completing this module, students have developed a strong foundation in Java programming and Object-Oriented Programming (OOPs). They learned what Java is, its history, important features, and how a Java program is written, compiled, and executed using the JVM. They also understood why Java is considered simple, secure, robust, portable, platform-independent, multithreaded, and dynamic.

Students also learned the core OOP concepts—Class, Object, Encapsulation, Inheritance, Polymorphism, and Abstraction—and understood how these concepts help create reusable, maintainable, scalable, and real-world-oriented applications.

At the end of this module, students should be able to:

- ✓ Explain the fundamentals and features of Java.
- ✓ Write, compile, and execute a basic Java program.
- ✓ Understand the Java source code → bytecode → JVM execution process.
- ✓ Create and use classes and objects.
- ✓ Apply Encapsulation for controlled access to data.
- ✓ Use Inheritance to achieve code reuse.
- ✓ Understand Polymorphism through method overloading and overriding.
- ✓ Understand Abstraction and hide unnecessary implementation details.
- ✓ Apply OOP principles to design programs that are easier to understand, maintain, reuse, and extend.

Overall, students have established the essential Java and OOP foundation required to progress toward advanced Java programming, frameworks, and real-world application development.